GUÍA PRÁCTICA LAB 03 Y PROMPTS OPTIMIZADOS • EDICIÓN TELECOMUNICACIONES (TELECOM NEXUS)

Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

TRYOLABS • GOOGLE ANTIGRAVITY WORKSHOP • LAB 03 (ES)

Lab 03: Telemetría OSS/BSS en Vivo con FastMCP, Lifecycle Hooks y Seguridad Gemini Enterprise

Duración: 50 minutos

Impartido por: **Tryolabs** (Impulsado por Google Antigravity & Gemini Enterprise)

Capacidades de Antigravity Practicadas: Model Context Protocol (**`mcp_config.json`** + FastMCP), Lifecycle Hooks (**`hooks.json`** **`PreToolUse`** y **`PostToolUse`**), Controles de Seguridad del Cliente Antigravity (Sandbox, Tool Execution Policy, Allowlists/Denylists) y Gobernanza en Google Cloud **Gemini Enterprise**.

Idioma / Language: Español | English • **Guía de Prompts:** PROMPTS.es.md | PROMPTS.md • **PDF:** ES | EN

1. Escenario y Arquitectura de Seguridad Empresarial

Los operadores de telecomunicaciones gestionan infraestructura crítica (gNodeBs 5G RAN, funciones de tasación CHF) e información regulada de suscriptores (**CPNI**: **`IMSI`**, **`MSISDN`**, **`IMEI`**). Conectar agentes de IA a bases de datos OSS/BSS requiere dos capas de gobernanza:

Capa 1: Gobernanza en Google Cloud y Gemini Enterprise (Plano Cloud)

Cuando los participantes autentican Antigravity con su licencia corporativa de **Gemini Enterprise**:

1. **IAM y Gestión de Licencias**: El acceso se controla mediante Google Cloud IAM (**`roles/cloudaicompanion.user`**) federado con el SSO corporativo (Workspace / Cloud Identity / Okta / Entra ID).
2. **Cloud Audit Logs (`cloudaicompanion.googleapis.com`)**: Los administradores auditan eventos **`Admin Activity`** y **`Data Access`** en **Google Cloud Logging** para trazabilidad y cumplimiento, sin registrar código propietario ni datos sensibles de suscriptores.
3. **Cero Entrenamiento y Perímetro VPC-SC**: El código, los prompts y la telemetría 3GPP/BSS **nunca** se utilizan para entrenar modelos fundacionales, cuentan con indemnización de propiedad intelectual de Google Cloud y pueden protegerse con **VPC Service Controls (VPC-SC)**.

Capa 2: Controles de Seguridad en el Cliente Antigravity y Workspace (Plano Local)

Dentro de **Antigravity 2.0** y **Antigravity IDE** (**`Settings`** -> **`Global`** y **`Project-Level Settings`**):

1. **Tool Execution Policy**: Permite elegir entre **`strict`**, **`request-review`**, **`proceed-in-sandbox`** o **`always-proceed`**.
2. **Terminal Sandbox**: Aísla los comandos de terminal ejecutados por el agente dentro de un sandbox a nivel de sistema operativo.
3. **Políticas de Acceso a Archivos e Internet**: Restringe **`Non-Workspace File Access`** (**deny**/**ask**) e **`Internet Access Policy`** (**deny**/**ask**), además de **`Command Allowlist / Denylist`** y **`Browser Allowlist`**.
4. **Lifecycle Hooks Determinísticos (`.agents/hooks.json`)**: Interceptan cada llamada a herramientas (**`PreToolUse`** / **`PostToolUse`**) mediante **`scripts/audit_hook.py`** para bloquear comandos SQL destructivos (**`DROP TABLE`**, **`TRUNCATE`**, **`DELETE FROM b2b_contracts`**) antes de que se ejecuten.

2. Preparación del Laboratorio

Ejecuta el instalador del Lab 03 (que aplica automáticamente las soluciones de los Labs 01 y 02):

```
./labs/lab-03-mcp-and-hooks/install.sh
```

3. Instrucciones Paso a Paso

Paso 3.1: Completar las 4 Herramientas FastMCP en `mcp_server/telecom_mcp.py`

Abre mcp_server/telecom_mcp.py. **`query_ran_cell_health`** ya está implementada. Usa Antigravity (ver PROMPTS.es.md) para implementar las tres funciones **`@mcp.tool()`** restantes:

1. **`calculate_contract_penalty_exposure(contract_id: str | None = None)`**
2. **`inspect_unbilled_cdrs(contract_id: str | None = None, roaming_only: bool = False)`** — **Importante**: Enmascara **`imsi`** y **`msisdn`** como **`masked_imsi`** y **`masked_msisdn`** (ej. **`3104***0001`**) para proteger la privacidad CPNI.
3. **`evaluate_slice_capacity(target_tac: str, requested_slice_sst: int, requested_bandwidth_mbps: int, max_latency_ms: float)`**

Paso 3.2: Registrar el Servidor MCP en `.agents/mcp_config.json`

Crea **`.agents/mcp_config.json`**:

```
{
  "mcpServers": {
    "telecom-nexus-mcp": {
      "command": "./.venv/bin/python",
      "args": ["mcp_server/telecom_mcp.py"],
      "env": {
        "PYTHONUNBUFFERED": "1"
      }
    }
  }
}
```

Paso 3.3: Configurar Hooks `PreToolUse` y `PostToolUse` en `.agents/hooks.json`

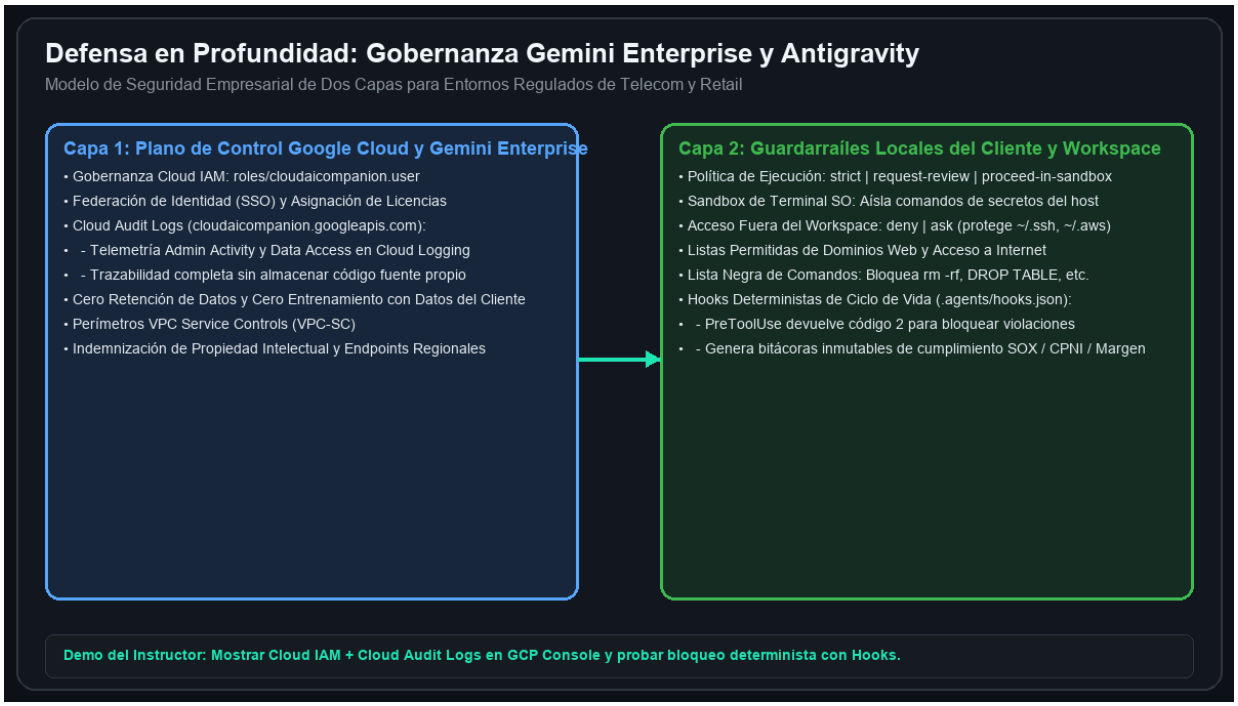
Inspecciona scripts/audit_hook.py y crea **`.agents/hooks.json`** para validar cada ejecución de **`run_command`** y registrar la auditoría en **`data/logs/hook_audit.log`**.

4. Verificación

Ejecuta el script de verificación del Lab 03:

```
./labs/lab-03-mcp-and-hooks/verify.sh
```

¿Necesitas la solución de referencia? Ejecuta **`./labs/lab-03-mcp-and-hooks/apply_solution.sh`**.



LAB 03 PROMPTS OPTIMIZADOS PARA COPIAR Y PEGAR (ES)
Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

Lab 03 — Guía de Prompts Optimizados para Antigravity (Español)

Idioma / Language: Español | English • **Guía del Lab:** README.es.md | README.md • **PDF:** ES | EN

Objetivo: Conectar **Google Antigravity** (gobernado por **Gemini Enterprise** e impartido por **Tryolabs**) a bases de datos Telecom OSS/BSS en vivo mediante **Model Context Protocol (FastMCP)** e imponer controles determinísticos de seguridad CPNI y de base de datos mediante **Lifecycle Hooks (`hooks.json`)**.

1. Prompt de Descubrimiento (@ Mentions)

```
@mcp_server/telecom_mcp.py @scripts/audit_hook.py @tests/test_lab03_mcp_hooks.py
Inspecciona el servidor FastMCP inicial en `mcp_server/telecom_mcp.py`, el validador de  \
  hooks de cumplimiento en `scripts/audit_hook.py` y las pruebas en `tests/test_lab03_mc \
  p_hooks.py`. Lista las 3 herramientas MCP pendientes de implementar en `telecom_mcp.py \
  ` y los esquemas exactos requeridos para `.agents/mcp_config.json` y `.agents/hooks.js \
  on`.
```

2. Prompt en Modo Planificación (`/plan`)

```
/plan Crea un plan de implementación para el Lab 03 que:
1. Complete `mcp_server/telecom_mcp.py` implementando:
   - `calculate_contract_penalty_exposure(contract_id: str | None = None)` usando `evalu \
  ate_contract_sla_compliance()` y `Decimal`.
   - `inspect_unbilled_cdrs(contract_id: str | None = None, roaming_only: bool = False)` \
   usando `rate_cdr_record()` y enmascarando `imsi` (`masked_imsi`) y `msisdn` (`masked_ \
  msisdn`) con `***` para cumplimiento CPNI.
   - `evaluate_slice_capacity(target_tac: str, requested_slice_sst: int, requested_bandw \
  idth_mbps: int, max_latency_ms: float)` retornando `READY_TO_ACTIVATE`, `REQUIRES_SLIC \
  E_REMEDIATION` o `REJECTED`.
2. Cree `.agents/mcp_config.json` registrando `telecom-nexus-mcp` por Stdio (`./.venv/bi \
  n/python mcp_server/telecom_mcp.py`).
3. Cree `.agents/hooks.json` vinculando `PreToolUse` (`matcher: "run_command"`) y `PostT \
  oolUse` (`matcher: "run_command|edit_file|write_file"`) con `scripts/audit_hook.py`.
```

3. Prompt de Edición Inline (`Cmd+I` / `Ctrl+I`)

Abre **`mcp_server/telecom_mcp.py`**, selecciona las tres funciones **`@mcp.tool()`** pendientes y presiona **`Cmd+I`** / **`Ctrl+I`**:

```
Implementa `calculate_contract_penalty_exposure`, `inspect_unbilled_cdrs` (enmascarando  \
  IMSI y MSISDN con `***` en `masked_imsi` y `masked_msisdn`) y `evaluate_slice_capacity \
  ` utilizando `src.services.sla_monitor`, `src.services.cdr_rating` y `src.database.fet \
  ch_all` para que pasen todas las pruebas en `tests/test_lab03_mcp_hooks.py`.
```

4. Prompt de Implementación Autónoma y Verificación

```
Implementa las 3 herramientas MCP pendientes en @mcp_server/telecom_mcp.py, crea `.agent \
  s/mcp_config.json` y `.agents/hooks.json`, y ejecuta `./labs/lab-03-mcp-and-hooks/veri \
  fy.sh`.
```

5. Prompt para Probar Herramientas MCP y Hooks de Seguridad

Una vez que **`verify.sh`** pase, prueba las herramientas MCP y el hook **`PreToolUse`** desde el Chat de Antigravity:

```
1. Usa nuestras herramientas Telecom MCP (o invoca las funciones en `mcp_server/telecom_ \
  mcp.py`) para inspeccionar `TAC-NY-001` slice `SST=2`, calcular la exposición a penali \
  dades de `CTR-B2B-1002`, e inspeccionar los CDRs de roaming sub-facturados con IMSIs e \
  nmascarados.
2. Prueba nuestro hook `PreToolUse` simulando una llamada a `scripts/audit_hook.py --eve \
  nt PreToolUse` con un comando SQL destructivo (`DROP TABLE b2b_contracts;`) y muestra  \
  cómo retorna `"decision": "deny"` y registra el evento en `data/logs/hook_audit.log`.
```

6. Prompts Avanzados de Seguridad y Gobernanza (`/boost`, `/grill-me`, `/schedule`)

Revisión Profunda de Arquitectura de Seguridad (`/boost`)

```
/boost Realiza un modelado de amenazas integral para Telecom OSS/BSS combinando nuestros \
   hooks locales `.agents/hooks.json`, el Terminal Sandbox de Antigravity y la gobernanz \
  a corporativa de Google Cloud Gemini Enterprise (`roles/cloudaicompanion.user`, Cloud  \
  Audit Logs en `cloudaicompanion.googleapis.com` y VPC Service Controls).
```

Monitoreo Programado de SLA y Hooks (`/schedule`)

```
/schedule Cada 10 minutos, ejecuta `query_ran_cell_health` en los 5 TACs y revisa `data/ \
  logs/hook_audit.log` en busca de ejecuciones denegadas.
```